



UNIVERSIDAD SANTO TOMÁS
PRIMER CLAUSTRO UNIVERSITARIO DE COLOMBIA
EDUCACIÓN ABIERTA Y A DISTANCIA
VIGILADA MINEDUCACIÓN



INTEGRACIÓN DE SISTEMAS DE INFORMACIÓN_55NI1
ACTIVIDAD 2

Docente:

CRISMAN MARTINEZ BARRERA

Nombre del estudiante:

DIANA MILENA MORENO ACEVEDO

CC 53121704

Universidad Santo Tomás

Ingeniería en Informática

Bogotá

2026

DESCRIPCIÓN GENERAL DEL PROYECTO

En este proyecto desarrollamos una solución de integración de sistemas de información a partir de un sistema de riego inteligente. Para esto, integramos diferentes componentes de hardware y software que trabajan en conjunto para capturar datos, procesarlos, almacenarlos y generar información que pueda ser consultada y monitoreada.

La solución cuenta con un ESP32-CAM, un sensor de humedad del suelo, un sensor de luz, un módulo de relé y una bomba de agua. El ESP32 se encarga de leer los sensores y, mediante el firmware, interpretar los valores obtenidos para determinar el estado de la humedad y la luminosidad. Cuando se detecta que el suelo está seco, el sistema activa la bomba durante un tiempo determinado. Además, el dispositivo genera los datos de telemetría y los envía al computador por medio de comunicación serial.

En el computador desarrollamos una aplicación en Python que recibe esta información y se encarga de validarla antes de procesarla. La aplicación también permite detectar datos fuera de los rangos establecidos, registrar eventos y errores, manejar la desconexión y reconexión del dispositivo y consultar algunos recursos del sistema operativo, como el uso de CPU, memoria RAM y almacenamiento.

Los datos que cumplen con las validaciones se almacenan en una base de datos MySQL y también se conservan en un archivo JSON, donde se mantiene un histórico de las lecturas recibidas. Por otra parte, los eventos importantes del sistema, como las desconexiones, reconexiones y datos inválidos, quedan registrados para poder hacer seguimiento al funcionamiento de la aplicación. Al finalizar la ejecución también se genera un reporte con los principales resultados obtenidos.

De esta manera, el proyecto integra las diferentes partes de la solución siguiendo el flujo:

Hardware → Firmware → Comunicación serial → Sistema operativo → Python → MySQL/JSON → Reporte

Con este flujo podemos seguir el recorrido de los datos desde el momento en que son obtenidos por los sensores hasta que son procesados, almacenados y convertidos en información que permite conocer el comportamiento del sistema.

ARQUITECTURA



La arquitectura del sistema está organizada en diferentes capas que trabajan de manera conjunta para llevar los datos desde los dispositivos físicos hasta su almacenamiento y posterior consulta. En la primera parte se encuentra el hardware, compuesto por el ESP32-CAM, los sensores y los elementos encargados de realizar la acción física del sistema. Sobre el ESP32 se encuentra el firmware, que se encarga de leer los sensores, interpretar sus valores y controlar la bomba de acuerdo con las condiciones establecidas.

La información generada por el dispositivo se envía mediante comunicación serial USB hacia el computador. Allí interviene el sistema operativo Windows, que proporciona los recursos necesarios para establecer la comunicación con el puerto COM y ejecutar la aplicación.

La siguiente capa corresponde a la aplicación desarrollada en Python, que recibe los datos enviados por el ESP32, los valida, procesa las lecturas, registra los eventos y controla situaciones como datos inválidos o pérdida de comunicación. Además, la aplicación consulta recursos del sistema operativo, como CPU, memoria y almacenamiento.

Finalmente, la información procesada se lleva a la capa de almacenamiento y generación de información, donde las telemetrías válidas se guardan en MySQL y en un archivo JSON, mientras que los eventos y errores se registran para facilitar el seguimiento del sistema. A partir de esta información también se genera un reporte de ejecución.

HARDWARE Y FIRMWARE

Componentes físicos:

Componente	Función en el sistema
ESP32	Controlador principal; ejecuta el firmware
Sensor de humedad de suelo (con placa comparadora)	Mide la humedad del suelo (salida analógica)
Módulo sensor de luz (fotorresistencia + comparador)	Mide la luz ambiental (salida digital)
Módulo relé de 5V (1 canal)	Activa y desactiva la bomba de agua
Bomba de agua sumergible 5V DC	Actuador de riego
Fuente externa 5V / batería	Alimentación independiente de la bomba, aislada del ESP32

Firmware (ESP32):

El firmware, escrito en Arduino/C++, se estructura en los siguientes bloques:

- ✓ Pines y configuración: define los pines de los sensores y el relé, el umbral de humedad seca (3000), el tiempo de riego (5 s) y el intervalo de lectura (15 s), y el identificador del dispositivo.

```
8 // ----- PINES -----
9 const int SENSOR_HUMEDAD = 13;
10 const int SENSOR_LUZ = 15;
11 const int RELE = 14;
12
13
14 // ----- CONFIGURACIÓN -----
15 const int UMBRAL_SECO = 3000;
16
17 const unsigned long TIEMPO_RIEGO = 5000; // 5 segundos
18 const unsigned long INTERVALO_LECTURA = 15000; // 15 segundos
19
20 const char* DEVICE_ID = "ESP32-CAM-001";
```

- ✓ setup(): inicializa el puerto serial, configura los pines, y apaga la bomba explícitamente antes de cualquier otra acción, como medida de seguridad.

```

30 // =====
31 // SETUP
32 // =====
33 void setup() {
34
35     Serial.begin(115200);
36
37     delay(1000);
38
39     // Configuración de pines
40     pinMode(SENSOR_HUMEDAD, INPUT);
41     pinMode(SENSOR_LUZ, INPUT);
42     pinMode(RELE, OUTPUT);
43
44     // Asegurar bomba apagada al iniciar
45     digitalWrite(RELE, LOW);

```

- ✓ `loop()`: controla el apagado de la bomba una vez transcurrido el tiempo de riego (independiente de la aplicación Python), y dispara una nueva lectura cada 15 segundos.

```

61 // =====
62 // LOOP PRINCIPAL
63 // =====
64 void loop() {
65
66     unsigned long ahora = millis();
67
68
69     // -----
70     // CONTROL DEL TIEMPO DE RIEGO
71     // -----
72     if (bombaEncendida) {
73
74         if (ahora - inicioRiego >= TIEMPO_RIEGO) {
75
76             digitalWrite(RELE, LOW);
77
78             bombaEncendida = false;
79
80             Serial.println("[EVENTO] RIEGO_FINALIZADO");
81             Serial.println("[BOMBA] OFF");
82             Serial.println();
83         }
84     }

```

- ✓ `leerSensores()`: lee la humedad (`analogRead`) y la luz (`digitalRead`); valida que la humedad esté en el rango físicamente posible (0–4095); clasifica el estado en SECO / MEDIO / HUMEDO y la luz en LUZ / OSCURIDAD; decide si debe iniciar el riego; arma y envía la telemetría en formato JSON por el puerto serial.

<pre> 99 // ===== 100 // LECTURA DE SENSORES 101 // ===== 102 void leerSensores() { 103 104 // ===== 105 // LECTURA REAL DEL SENSOR 106 // ===== 107 108 int humedad = analogRead(SENSOR_HUMEDAD); 109 110 int luzDigital = digitalRead(SENSOR_LUZ); 111 112 113 // ----- 114 // VALIDACIÓN DEL VALOR DE HUMEDAD 115 // ----- 116 117 if (humedad < 0 humedad > 4095) { 118 119 Serial.println("[ALERTA] VALOR_HUMEDAD_FUERA_DE_RANGO"); 120 121 return; 122 } 123 </pre>	<pre> 125 // ----- 126 // CLASIFICACIÓN DE HUMEDAD 127 // ----- 128 129 String estadoHumedad; 130 131 132 if (humedad > UMBRAL_SECO) { 133 134 estadoHumedad = "SECO"; 135 136 } 137 else if (humedad >= 1800) { 138 139 estadoHumedad = "MEDIO"; 140 141 } 142 else { 143 144 estadoHumedad = "HUMEDO"; 145 146 } </pre>
--	---

```

148 // -----
149 // CLASIFICACIÓN DE LUZ
150 // -----
151 //
152 // En nuestro módulo:
153 //
154 // 0 = LUZ
155 // 1 = OSCURIDAD
156 //
157 // -----
158
159 String estadoLuz;
160
161
162 if (luzDigital == 0) {
163     estadoLuz = "LUZ";
164 }
165 else {
166     estadoLuz = "OSCURIDAD";
167 }
168
169 }
170

```

- ✓ `iniciarRiego()`: activa el relé, registra el momento de inicio, y deja constancia del evento en el puerto serial.

```

252 // =====
253 // INICIAR RIEGO
254 // =====
255 void iniciarRiego() {
256     bombaEncendida = true;
257     inicioRiego = millis();
258     // Activar relé
259     digitalWrite(RELE, HIGH);
260     Serial.println();
261     Serial.println("[ALERTA] HUMEDAD_BAJA");
262     Serial.println("[EVENTO] RIEGO_INICIADO");
263     Serial.println("[BOMBA] ON");
264     Serial.println();
265 }
266

```

```

206 // -----
207 // DECISIÓN DE RIEGO
208 // -----
209
210 if (estadoHumedad == "SECO" && !bombaEncendida) {
211     iniciarRiego();
212 }
213

```

Responsabilidades del firmware:

- Inicializar comunicación serial y pines.
- Leer humedad y luz.
- Clasificar la humedad como SECO, MEDIO o HUMEDO.
- Determinar si debe iniciar el riego.
- Controlar el relé durante el tiempo configurado.
- Construir la telemetría JSON.
- Enviar la información al computador.

APLICACIÓN PYTHON

(Ver código completo [aquí](#))

La aplicación se organiza en funciones independientes, cada una con una responsabilidad clara:

- ❖ **conectar_mysql():** abre la conexión a la base de datos con manejo de errores; si falla, retorna None sin detener el flujo.
- ❖ **registrar_evento():** inserta cada evento del sistema (conexión, error, dato rechazado, riego) en la tabla eventos de MySQL, es el mecanismo de logging estructurado del sistema.
- ❖ **guardar_telemetria():** inserta cada lectura válida en la tabla telemetria.
- ❖ **guardar_json():** mantiene un historial de hasta 1000 registros en un archivo JSON, con manejo de errores de escritura (por ejemplo, permisos denegados).
- ❖ **validar_datos():** valida que estén los campos requeridos, que la humedad sea un entero dentro de rango, y que los estados de humedad, luz y bomba sean valores permitidos.
- ❖ **consultar_recursos_sistema() / mostrar_recursos_sistema():** consulta, mediante la librería psutil, el uso de CPU, RAM y disco del sistema operativo, además del consumo específico del proceso Python.
- ❖ **conectar_esp32():** abre el puerto serial de forma segura, retornando None si no está disponible, para permitir reintentos posteriores.
- ❖ **generar_reporte_ejecucion():** genera, al finalizar la ejecución, un archivo de texto con estadísticas completas.

El bloque principal integra todo: se conecta a MySQL y al ESP32, y en un bucle continuo consulta recursos del sistema, reintenta la conexión con el ESP32 si se pierde, procesa cada línea recibida distinguiendo mensajes informativos, eventos de riego y telemetría, valida y persiste los datos válidos, y registra los rechazados. Cualquier excepción de puerto serial se captura puntualmente para no detener la aplicación; el bloque finally siempre genera el reporte y cierra las conexiones de forma ordenada.

PERSISTENCIA Y REPORTES

Los datos válidos se persisten en tres formas simultáneas:

- ✓ **Base de datos MySQL:** tabla telemetria (lecturas de sensores) y tabla eventos (bitácora estructurada del sistema).

Tabla	Propósito	Información
telemetria	Persistencia de datos	fecha_hora, device_id, humedad_raw, humedad_estado, luz y bomba
eventos	Trazabilidad	fecha_hora, device_id, nivel, evento y resultado

- ✓ Archivo JSON histórico (telemetry_actual.json): El archivo telemetry_actual.json mantiene una lista de las telemetrías recibidas. Se conserva un historial de múltiples registros y se limita su tamaño para evitar un crecimiento indefinido.

```
1 [{"fecha_hora": "2026-09-23 22:39:54",
2   "device_id": "ESP32-CAM-001",
3   "humidity_raw": 2365,
4   "humidity_state": "MEDIO",
5   "light": "LUZ",
6   "pump": "OFF"}],
7
8 [{"fecha_hora": "2026-09-23 22:40:09",
9   "device_id": "ESP32-CAM-001",
10  "humidity_raw": 2349,
11  "humidity_state": "MEDIO",
12  "light": "LUZ",
13  "pump": "OFF"}],
14
15 ],
16
17 ]
```

- ✓ Reporte de ejecución (reporte_ejecucion.txt): Al finalizar con Ctrl+C, la aplicación genera reporte_ejecucion.txt. El archivo resume el dispositivo, puerto, inicio, fin, duración, mensajes recibidos, telemetrías válidas, datos rechazados, inserciones en MySQL, registros JSON, riegos, desconexiones, reconexiones y recursos del sistema.

≡ reporte_ejecucion.txt

```
1 =====
2 | REPORTE RESUMIDO DE EJECUCIÓN - RF08
3 | SISTEMA DE RIEGO INTELIGENTE
4 =====
5 Dispositivo: ESP32-CAM-001
6 Puerto serial: COM3
7 Inicio: 2026-09-26 20:11:16
8 Fin: 2026-09-26 20:18:25
9 Duración: 00:07:09
10
11 --- TELEMETRÍA ---
12 Mensajes DATA recibidos: 29
13 Telemetrías válidas: 29
14 Datos inválidos rechazados: 0
15 JSON inválidos recibidos: 0
16 Inserciones en MySQL: 29
17 Registros actuales en JSON: 252
18
19 --- EVENTOS ---
20 Riegos iniciados: 0
21 Riegos finalizados: 0
22 Desconexiones detectadas: 0
23 Reconexiones realizadas: 0
24
25 --- RECURSOS DEL SISTEMA AL FINALIZAR ---
26 CPU total: 5.9%
27 RAM utilizada: 74.7%
28 Disco ocupado: 33.9%
29 Disco libre: 330.67 GB
30 CPU del proceso Python: 0.0%
31 RAM del proceso Python: 35.68 MB
32
33 --- RESULTADO ---
34 La ejecución finalizó de forma controlada por el usuario.
35 Los datos válidos fueron persistidos en MySQL y JSON.
36 Los datos inválidos fueron rechazados y registrados como eventos.
```

FALLOS CONTROLADOS — DETALLE

Dato fuera de rango:

Campo	Descripción
Causa	Se envió manualmente un valor de humidity_raw = 5000, fuera del rango físico.
Síntoma	El dato no se almacena en la tabla telemetria porque no supera la validación.
Detección	La función validar_datos() detecta que el valor está fuera de rango y devuelve el motivo “Humedad fuera de rango” .
Respuesta del sistema	El dato es rechazado, se registra el evento INVALID_DATA en la tabla eventos y la aplicación continúa ejecutándose normalmente
Evidencia	En la consola se observa el valor 5000, el mensaje “Humedad fuera de rango” , la validación del dato y el registro INVALID_DATA.
Propuesta de mejora	Generar una alerta si se acumulan varios rechazos consecutivos, ya que esto podría indicar un problema con el sensor o con los datos recibidos.

```
PS D:\Mega\Ingeniería en Informática\2026-2 final\Integración de sistemas de información\Actividad 2> python sistema_riego_reconexion.py
```

```
[RECURSOS DEL SISTEMA OPERATIVO]
```

```
=====
```

```
CPU total: 5.1%
```

```
RAM total utilizada: 52.5%
```

```
Disco ocupado: 33.9%
```

```
Disco libre: 330.68 GB
```

```
[PROCESO PYTHON]
```

```
CPU del proceso: 0.0%
```

```
RAM del proceso: 34.57 MB
```

```
[ESP32] [PRUEBA] Enviando dato fuera de rango a Python...
```

```
[TELEMETRÍA] Datos recibidos:
```

```
{"device_id": "ESP32-CAM-001", "humidity_raw": 5000, "humidity_state": "SECO", "light": "LUZ", "pump": "OFF"}
```

```
[ALERTA] Humedad fuera de rango: 5000
```

```
[VALIDACIÓN] Datos rechazados.
```

```
[EVENTO MYSQL] INVALID_DATA -> Humedad fuera de rango
```

Pérdida de comunicación:

Campo	Descripción
Causa	Se desconectó físicamente el cable USB del ESP32 durante la ejecución.
Síntoma	Se interrumpe la recepción de telemetría y se presenta un error de comunicación con el puerto serial.
Detección	La aplicación captura la excepción asociada al puerto serial, en este caso un PermissionError al intentar acceder al dispositivo.
Respuesta del sistema	Se registra el evento DEVICE_DISCONNECTED, la aplicación continúa ejecutándose y comienza a intentar nuevamente la conexión cada 5 segundos, sin detener el proceso.
Evidencia	En la consola se observa el error de comunicación, el registro DEVICE_DISCONNECTED y los intentos de reconexión cada 5 segundos.

Campo	Descripción
Propuesta de mejora	Notificar al usuario si la desconexión se mantiene durante un tiempo prolongado y registrar el tiempo total que permaneció sin comunicación.

```
PS D:\Mega\Ingeniería en Informática\2026-2 final\Integración de sistemas de información\Actividad 2> python sistema_riego_reconexion.py
[JSON] Telemetría agregada. Registros almacenados: 253
[ESP32] -----

ERROR DE COMUNICACIÓN CON EL ESP32
ClearCommError failed (PermissionError(13, 'Acceso denegado.', None, 5))
[EVENTO MYSQL] DEVICE_DISCONNECTED -> Error serial: ClearCommError failed (PermissionError(13, 'Acceso denegado.', None, 5))
[RECONEXIÓN] Python continuará ejecutándose.
[RECONEXIÓN] Próximo intento en 5 segundos.
[RECONEXIÓN] Intentando conectar ESP32...
[RECONEXIÓN] ESP32 todavía no disponible.
[RECONEXIÓN] Intentando conectar ESP32...
[RECONEXIÓN] ESP32 todavía no disponible.
```

Archivo JSON inaccesible:

Campo	Descripción
Causa	Se envió desde el ESP32 un mensaje con formato JSON incorrecto para comprobar la validación de los datos recibidos por Python.
Síntoma	La aplicación recibe el mensaje, pero no puede interpretarlo como un JSON válido.
Detección	La aplicación Python detecta el error al intentar procesar el mensaje recibido desde el ESP32 y lo clasifica como dato inválido.
Respuesta del sistema	El dato inválido es rechazado y no se procesa como telemetría. Se registra el evento INVALID_DATA indicando que se recibió un JSON inválido y la aplicación continúa ejecutándose normalmente.
Evidencia	En la consola se observa el mensaje [ERROR] JSON inválido recibido desde ESP32 y el registro [EVENTO MYSQL] INVALID_DATA → JSON inválido.
Propuesta de mejora	Implementar validaciones adicionales sobre la estructura de los mensajes y registrar información más detallada sobre el origen y tipo de error para facilitar su diagnóstico.

```
PS D:\Mega\Ingeniería en Informática\2026-2 final\Integración de sistemas de información\Actividad 2> python sistema_riego_reconexion.py
=====
CPU total: 3.7%
RAM total utilizada: 53.8%
Disco ocupado: 33.9%
Disco libre: 330.67 GB

[PROCESO PYTHON]
CPU del proceso: 0.0%
RAM del proceso: 34.79 MB

[ESP32] -----
[ESP32] DEVICE_ID: ESP32-CAM-001
[ESP32] HUMEDAD_RAW: 2412
[ESP32] HUMEDAD_ESTADO: MEDIO
[ESP32] LUZ: LUZ
[ESP32] BOMBA:
[ESP32] [PRUEBA] Enviando JSON mal formado a Python...

[TELEMETRÍA] Datos recibidos:
{"device_id":"ESP32-CAM-001","humidity_raw":2448,"humidity_state":"MEDIO","light":"LUZ","pump":"OFF"}
[ERROR] JSON inválido recibido desde ESP32.
[EVENTO MYSQL] INVALID_DATA -> JSON inválido
```

MATRIZ DE PRUEBAS

Se diseñaron y ejecutaron 10 pruebas, superando el mínimo de 2 fallos controlados solicitado.

MATRIZ DE PRUEBAS DEL SISTEMA				
	Prueba	procedimiento	Resultado esperado	Resultado obtenido
T01	Telemetría normal	Ejecutar el sistema con el ESP32 conectado y recibir lecturas periódicas de los sensores.	Python recibe, valida y almacena los datos correctamente.	Se recibieron valores como 2128 y 2192, clasificados como MEDIO, con luz LUZ y bomba OFF. Los datos fueron almacenados en MySQL y JSON.
T02	Dato fuera de rango	Enviar temporalmente humidity_raw = 5000, superando el rango válido del ADC.	Python debe rechazar el dato y registrar el evento.	Python mostró Humedad fuera de rango: 5000 y registró INVALID_DATA en MySQL. El dato no fue almacenado como telemetría válida.
T03	Persistencia en MySQL	Recibir telemetrías válidas durante la ejecución.	Cada dato válido debe almacenarse en telemetría.	Se verificaron registros en la tabla telemetría, incluyendo valores de humedad, estado, luz y bomba.
T04	Persistencia JSON	Recibir varias telemetrías válidas.	Los datos deben agregarse al archivo JSON histórico.	El archivo telemetría_actual.json acumuló registros, durante una ejecución se observaron 11 registros almacenados.
T05	Consulta de recursos del SO	Ejecutar la aplicación Python y consultar recursos del sistema.	Obtener información de CPU, RAM y almacenamiento.	Python mostró CPU total, RAM utilizada, disco ocupado/libre y consumo del proceso Python.
T06	Pérdida de comunicación	Desconectar temporalmente la comunicación USB/serial del ESP32.	Python debe detectar la pérdida y continuar ejecutándose.	Se registró DEVICE_DISCONNECTED y Python permaneció activo intentando recuperar la comunicación.
T07	Reconexión	Restaurar la conexión del ESP32 después de una desconexión.	Python debe recuperar automáticamente la comunicación y continuar recibiendo datos.	Se registró la reconexión y posteriormente se reanudó la recepción y almacenamiento de telemetría.
T08	Reporte de ejecución	Ejecutar el sistema y finalizar mediante Ctrl+C.	Generar automáticamente un reporte resumido.	Se generó reporte_ejecucion.txt con duración, telemetrías, eventos, errores y recursos del sistema.
T09	JSON mal formado	Enviar temporalmente un mensaje DATA con estructura JSON incompleta.	Python debe rechazarlo sin detener la aplicación y registrar el error.	Python mostró JSON inválido recibido desde ESP32 y registró INVALID_DATA -> JSON inválido en MySQL.
T10	Archivo JSON inaccesible	Sustituir temporalmente el archivo JSON por una carpeta con el mismo nombre.	Python debe detectar el fallo de escritura, registrar el error y continuar funcionando.	MySQL almacenó correctamente la telemetría, Python detectó Permission denied y registró JSON_WRITE_ERROR. La aplicación continuó ejecutándose.

ANÁLISIS Y RESPUESTAS AL ANEXO 3

Pregunta 1. ¿Qué responsabilidad pertenece al hardware y cuál al firmware? ¿Dónde se produce realmente cada transformación de datos?

El hardware realiza la medición física mediante los sensores y ejecuta físicamente acciones como el control de la bomba. El firmware interpreta las señales, clasifica la humedad y controla el relé. La primera transformación ocurre en el firmware cuando una lectura numérica se convierte en un estado como SECO, MEDIO o HUMEDO. Después, Python transforma el mensaje JSON recibido en datos estructurados que puede validar y almacenar.

Pregunta 2. ¿Qué servicios del sistema operativo utiliza la aplicación Python?

La aplicación utiliza el puerto serial para comunicarse con el ESP32 y servicios relacionados con el sistema de archivos, CPU, memoria, almacenamiento y procesos. La biblioteca psutil permite consultar los recursos del sistema y del proceso Python.

Pregunta 3. ¿Qué ocurre si el sistema operativo revoca permisos sobre un recurso utilizado por la aplicación?

La operación puede generar un error, por ejemplo `PermissionError` al intentar escribir el JSON. La aplicación debe capturar el error, registrarlo y continuar funcionando cuando sea posible. Durante el desarrollo se contempló la escritura del JSON como un punto donde un cambio de permisos puede afectar la persistencia sin necesariamente detener la recepción de telemetría.

Pregunta 4. ¿Qué ventajas y limitaciones tiene Python frente a C/C++ o Rust?

Python facilita el desarrollo de la aplicación de integración porque dispone de bibliotecas para comunicación serial, bases de datos, archivos y monitoreo. Su principal limitación frente a C/C++ o Rust es el menor control de bajo nivel y el mayor consumo relativo de recursos. Por eso en el proyecto Python se utiliza en el computador, mientras el firmware se desarrolla en C++.

Pregunta 5. ¿Qué parte debería ejecutarse en firmware y cuál en software de aplicación?

El firmware debe realizar las tareas que dependen directamente del dispositivo y requieren una respuesta local: lectura de sensores, clasificación básica, control de la bomba y generación de telemetría. La aplicación de software debe realizar validación, persistencia, reportes, logs, monitoreo y recuperación de comunicación. Esta separación reduce el acoplamiento y mantiene responsabilidades claras.

Pregunta 6. ¿Cómo cambiaría la arquitectura si se conectaran 100 dispositivos simultáneamente?

Con 100 dispositivos, la comunicación serial individual dejaría de ser conveniente. La arquitectura podría migrar a comunicación por red, utilizando por ejemplo MQTT o HTTP, con un identificador único por dispositivo. Los datos podrían llegar a un broker o API, pasar por un servicio de procesamiento y terminar en una base de datos central.

100 ESP32 → Red → Broker/API → Servicio de procesamiento → Base de datos → Dashboard/Reportes

Pregunta 7. ¿Qué riesgos de seguridad aparecen al recibir datos externos o ejecutar comandos del sistema?

Existen riesgos de datos manipulados, valores mal formados, inyección de información, ejecución de comandos no confiables y consumo excesivo de recursos. Para reducirlos se deben validar los datos, limitar permisos, evitar ejecutar directamente información recibida y registrar los errores.

Pregunta 8. ¿Cómo garantizaría integridad y trazabilidad?

La integridad se controla validando los datos antes de almacenarlos. La trazabilidad se obtiene mediante fecha y hora, identificador del dispositivo, datos recibidos, resultado de validación, eventos y errores. La persistencia en MySQL y el historial JSON permiten conservar evidencia de las telemetrías procesadas.

Pregunta 9. ¿Qué diferencias podrían presentarse entre Windows y Linux?

La lógica general de Python puede conservarse, pero cambian aspectos como la identificación del puerto serial, rutas de archivos, permisos y administración de dispositivos. En este prototipo se utiliza Windows y COM3; en Linux normalmente el dispositivo tendría una identificación diferente y podrían requerirse permisos específicos para acceder al puerto.

Pregunta 10. ¿Cómo diseñaría una estrategia de actualización del firmware sin afectar el servicio?

En producción se recomienda una actualización controlada. Primero se prueba la nueva versión en un grupo pequeño, se verifica su funcionamiento y después se amplía progresivamente el despliegue. Debe conservarse una versión anterior para recuperación. En una evolución del sistema también podría implementarse OTA (Over-The-Air) para reducir la intervención física.

Seguridad, confiabilidad y mantenibilidad

- Validar toda información proveniente del dispositivo antes de persistirla.
- Separar responsabilidades entre firmware y aplicación.
- No ejecutar comandos contruidos a partir de datos externos.
- Utilizar permisos mínimos para archivos y servicios.
- Registrar eventos relevantes para facilitar diagnóstico.
- Mantener una estrategia de recuperación ante pérdida de comunicación.

Escalabilidad propuesta

La arquitectura actual es adecuada para un prototipo de un dispositivo. Para una solución de mayor escala se recomienda separar adquisición, transporte, procesamiento y almacenamiento. Cada dispositivo debe contar con un identificador único y la plataforma debe soportar concurrencia, colas de mensajes y una base de datos preparada para múltiples fuentes.

Pregunta 11. ¿Qué pruebas adicionales aplicaría antes de llevar el prototipo a producción?

Antes de producción se deberían realizar pruebas prolongadas y pruebas de condiciones de falla. Entre ellas:

- Pérdida de alimentación del ESP32 y recuperación.
- Reinicio inesperado del dispositivo.
- Sensor desconectado o con lecturas anómalas.
- Pérdida prolongada de comunicación.
- Almacenamiento lleno o no disponible.
- Múltiples dispositivos enviando información simultáneamente.
- Pruebas de carga y estabilidad.
- Pruebas de seguridad y control de permisos.
- Pruebas ambientales del sensor y del sistema de riego.

Pregunta 12. ¿Qué componente sería el principal punto de falla y cómo lo convertiría en un componente tolerante a fallos?

Un punto importante de falla es la comunicación entre el ESP32 y Python. Una pérdida de comunicación puede impedir temporalmente la recepción de telemetría. Para reducir el impacto se implementó detección de desconexión, registro del evento y reconexión automática. La aplicación continúa ejecutándose mientras el dispositivo no está disponible y retoma la recepción cuando se recupera.

CONCLUSIONES

El proyecto permitió implementar una integración completa entre un dispositivo físico, firmware, sistema operativo y aplicación de software. El ESP32-CAM realiza la adquisición y el procesamiento inicial de los datos, mientras Python se encarga de la validación, persistencia, observabilidad y manejo de errores.

La comunicación serial permitió establecer el flujo entre el dispositivo y el computador. La información válida se almacenó en MySQL y en un historial JSON, mientras los eventos y errores quedaron registrados para facilitar la trazabilidad.

Las pruebas de datos fuera de rango, JSON inválido, desconexión y reconexión demostraron que la aplicación puede responder ante diferentes condiciones de falla sin detener completamente el proceso. El reporte complementa las evidencias al consolidar los resultados de cada ejecución.